

Phase 5

Multi-Device + Advanced Testing

Milestones M25 - M28 | Weeks 29 - 36
Orchestrated Sandbox Coordination

SUMMARY

Phase 5 extends Probato from single-browser test execution to coordinated multi-sandbox orchestration. Four milestones build a foundational orchestrator (M25), cross-device messaging verification (M26), call flow and background state testing (M27), and Stripe payment flow validation (M28). The architecture uses a database-backed event bus with synchronization barriers to coordinate multiple browser instances within Vercel serverless constraints.

M25

**Multi-Sandbox
Orchestrator**

M26

**Cross-Device
Messaging**

M27

**Call Flow &
Background**

M28

**Payment Flow
Testing**

Table of Contents

1. Executive Summary	3
2. Phase 5 Overview	3
2.1 Objectives	3
2.2 Architecture Considerations	3
2.3 Serverless Constraints and Solutions	4
3. M25 - Multi-Sandbox Orchestrator	4
3.1 Objective	4
3.2 Data Models	5
3.3 Core Agent: Multi-Device Orchestrator	6
3.4 API Routes	7
3.5 UI Component: Orchestrator Panel	7
3.6 QA Success Criteria	7
4. M26 - Cross-Device Messaging and Notification Testing	8
4.1 Objective	8
4.2 Test Scenarios	8
4.3 Data Model Extension	9
4.4 Core Agent: Messaging Tester	9
4.5 API Routes	9
4.6 UI Component: Messaging Test Panel	10
4.7 QA Success Criteria	10
5. M27 - Call Flow and Background State Testing	10
5.1 Objective	11
5.2 Test Scenarios	11
5.3 Data Model Extension	11

5.4 Core Agent: Call Flow Tester	11
5.5 API Routes	12
5.6 UI Component: Call Flow Test Panel	12
5.7 QA Success Criteria	12
6. M28 - Payment Flow Testing	13
6.1 Objective	13
6.2 Test Scenarios	13
6.3 Data Model Extension	14
6.4 Core Agent: Payment Flow Tester	14
6.5 API Routes	14
6.6 UI Component: Payment Flow Test Panel	15
6.7 QA Success Criteria	15
7. Architecture Deep-Dive	15
7.1 Multi-Sandbox Coordination Pattern	15
7.2 Synchronization Protocol	16
7.3 Event Bus Design	16
7.4 Credit Cost Model	17
8. Prisma Schema Additions Summary	17
9. API Route Map	18
10. UI Component Inventory	19
11. Dependency and Risk Analysis	19
11.1 Dependencies	19
11.2 Risk Assessment	20
12. Timeline	20

1. Executive Summary

Phase 5 introduces **Multi-Device + Advanced Testing** to Probato, extending the platform from single-browser test execution to coordinated multi-sandbox orchestration. This phase enables Probato to test real-world cross-device scenarios including messaging between users, voice/video call flows, background notification delivery, and payment checkout workflows. These capabilities address a critical gap in automated testing: most existing tools can only test a single browser session in isolation, whereas modern applications increasingly require coordination between multiple simultaneous users and devices.

The phase is organized into four milestones (M25-M28) spanning eight weeks of development. M25 builds the foundational multi-sandbox orchestrator with a synchronization protocol and shared state machine. M26 leverages this infrastructure to test cross-device messaging and notification delivery. M27 extends the orchestrator for call flow testing with audio verification and background-state simulation. M28 completes the phase with end-to-end payment flow testing using Stripe test mode. Each milestone follows the established Probato patterns: Prisma data models, credit-based billing, LLM-powered analysis with 3-tier fallback, and interactive dashboard UI panels.

A key architectural challenge for Phase 5 is adapting multi-device orchestration to Vercel serverless constraints. Rather than maintaining persistent long-running processes, the orchestrator uses a database-backed event bus with short-lived serverless invocations that poll for synchronization points. This approach preserves the existing deployment model while enabling coordinated multi-browser scenarios through carefully designed state machines and sync primitives stored in PostgreSQL.

2. Phase 5 Overview

2.1 Objectives

The primary objective of Phase 5 is to enable Probato to execute coordinated test scenarios across multiple browser instances simultaneously. This requires solving three fundamental challenges: first, provisioning and managing multiple sandboxed browser environments within the existing serverless architecture; second, establishing a reliable synchronization protocol that allows separate browser sessions to coordinate their actions at defined points; and third, building intelligent test agents that can reason about cross-device state and verify behaviors that span multiple user sessions.

Secondary objectives include: extending the credit billing model to cover multi-device testing costs (which are higher due to multiple browser instances); building intuitive UI panels that visualize the state of parallel sandbox sessions; and ensuring the orchestrator degrades gracefully when browser resources are limited, falling back to sequential execution where necessary.

2.2 Architecture Considerations

The current Probato architecture runs on Vercel serverless with a maximum duration of 60 seconds per invocation. Browser automation uses Puppeteer with a 4-tier provider chain (Browserless cloud,

WebSocket endpoint, Sparticuz/Chromium for serverless, local Puppeteer fallback). All state is persisted in Neon PostgreSQL via Prisma ORM. Phase 5 must work within these constraints while introducing multi-browser coordination. The key insight is that true real-time synchronization between serverless functions is not possible through direct communication; instead, the orchestrator uses the database as a coordination layer with optimistic locking and polling-based sync points.

Each orchestrated session spawns multiple browser instances (2-4 typically) that operate as "agents" with assigned roles (e.g., Agent A = sender, Agent B = receiver). These agents communicate through a shared event table in PostgreSQL, writing events that other agents poll for. Synchronization barriers ensure agents wait for each other at critical points (e.g., Agent A waits for Agent B to be ready before sending a message). This pattern is analogous to how distributed systems implement consensus, simplified for the testing context where the orchestrator has complete control over all participants.

2.3 Serverless Constraints and Solutions

Constraint	Impact	Solution
60s max duration per invocation	Multi-device scenarios may exceed this limit	Chunk scenarios into phases; each phase is a separate invocation with DB-backed state continuation
No persistent connections between functions	Agents cannot directly communicate	Database event bus with polling; agents write/read events via Prisma
Memory limits (1024MB default)	Multiple browsers consume significant RAM	Max 2 concurrent browser instances per invocation; sequential fallback for 3+ agents
Cold start latency	Orchestrator startup delay	Pre-warm browser instances; reuse Browserless connections across invocations

Table 1: Serverless Constraints and Solutions

3. M25 - Multi-Sandbox Orchestrator

3.1 Objective

Build the foundational multi-sandbox orchestration layer that can provision, coordinate, and manage multiple browser instances running simultaneously. This milestone creates the infrastructure that all subsequent Phase 5 milestones depend on. The orchestrator handles sandbox lifecycle (create, run, teardown), synchronization primitives (barriers, events, shared state), and resource management

(browser instance pooling, memory budgeting). It exposes a high-level API that allows test scenarios to define agent roles, assign actions to each agent, and specify synchronization points where agents must wait for each other.

3.2 Data Models

Three new Prisma models are introduced to support multi-sandbox orchestration. The **OrchestratedSession** model represents a complete multi-device test run, tracking the overall status, participating agents, scenario configuration, and aggregated results. The **SandboxInstance** model represents an individual browser sandbox within a session, tracking its assigned role, current state, and individual test results. The **SyncEvent** model implements the event bus, recording synchronization events that agents produce and consume to coordinate their actions.

```
model OrchestratedSession {
  id String @id @default(cuid())
  projectId String
  project Project @relation(fields: [projectId], references: [id])
  scenarioType String // "messaging" | "call" | "payment" | "custom"
  status String @default("pending") // pending | running | completed | failed
  config Json // Scenario configuration (agents, sync points, etc.)
  overallScore Float? // 0-100 aggregated score
  agentResults Json? // Per-agent results
  summary String? // AI-generated summary
  findings Json? // Aggregated findings across all agents
  startedAt DateTime?
  completedAt DateTime?
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
  sandboxes SandboxInstance[]
  syncEvents SyncEvent[]
  @@index([projectId])
  @@index([status])
  @@map("orchestrated_sessions")
}

model SandboxInstance {
  id String @id @default(cuid())
  sessionId String
  session OrchestratedSession @relation(fields: [sessionId], references: [id],
  onDelete: Cascade)
  agentRole String // "sender" | "receiver" | "caller" | "callee" | "payer" |
  "merchant"
  browserId String? // Browser instance identifier
  status String @default("provisioning") // provisioning | ready | running | done |
  error
  actions Json? // Assigned action sequence
```

```

results Json? // Step-by-step results
score Float? // 0-100 per-agent score
errorLog String? // Error details if failed
startedAt DateTime?
completedAt DateTime?
createdAt DateTime @default(now())
updatedAt DateTime @updatedAt
@@index([sessionId])
@@map("sandbox_instances")
}
model SyncEvent {
id String @id @default(cuid())
sessionId String
session OrchestratedSession @relation(fields: [sessionId], references: [id],
onDelete: Cascade)
eventType String // "barrier" | "signal" | "state_update" | "error"
sourceAgent String // Agent role that produced the event
targetAgent String? // Target agent (null = broadcast)
payload Json? // Event data
consumed Boolean @default(false)
consumedAt DateTime?
createdAt DateTime @default(now())
@@index([sessionId, eventType])
@@index([sessionId, targetAgent, consumed])
@@map("sync_events")
}

```

3.3 Core Agent: Multi-Device Orchestrator

The **MultiDeviceOrchestrator** class serves as the central coordination engine. It is responsible for provisioning browser instances for each agent role, executing action sequences in parallel, managing synchronization barriers, and aggregating results. The orchestrator follows a lifecycle pattern: **plan** (parse scenario, assign roles) -> **provision** (create browser instances) -> **execute** (run agent actions with sync) -> **aggregate** (collect results, score) -> **teardown** (close browsers, write summary).

The file `src/lib/agent/multi-device-orchestrator.ts` implements this class with the following key methods:

- **runScenario(config)**: Main entry point. Parses scenario config, provisions sandboxes, executes the orchestration, and returns aggregated results.
- **provisionSandbox(role, actions)**: Creates a browser instance for a given agent role, assigns its action sequence, and marks it as ready in the database.
- **executeAgent(sandboxId)**: Runs an individual agent action sequence within its browser context, respecting sync barriers.

- **signalEvent(eventType, sourceAgent, payload)**: Writes a sync event to the database event bus, signaling other agents.
- **waitForEvent(eventType, targetAgent, timeout)**: Polls the database for a matching unconsumed event, consuming it when found. Implements exponential backoff with configurable timeout.
- **barrier(barrierName, agentCount)**: Implements a synchronization barrier. Each agent signals arrival; execution resumes only when all agents have arrived.
- **aggregateResults(sessionId)**: Collects results from all sandbox instances, runs LLM analysis on cross-agent findings, and computes an overall session score.
- **teardown(sessionId)**: Gracefully closes all browser instances, updates session status, and triggers any post-session hooks.

3.4 API Routes

Method	Route	Description
POST	/api/orchestrator/sessions	Create and start a new orchestrated session (deducts credits)
GET	/api/orchestrator/sessions	List orchestrated sessions for a project with pagination
GET	/api/orchestrator/sessions/[id]	Get session detail with sandbox states and sync events
POST	/api/orchestrator/sessions/[id]/abort	Abort a running session and teardown all sandboxes

Table 2: M25 API Routes

3.5 UI Component: Orchestrator Panel

The **OrchestratorPanel** component provides a visual dashboard for multi-sandbox sessions. It displays an overall session score as a circular gauge, a timeline view of agent actions across parallel sandboxes, and a sync event log showing barrier arrivals and signal exchanges. Each sandbox is shown as a card with its role, status badge (provisioning/ready/running/done/error), individual score, and expandable action result list. The panel supports filtering by scenario type (messaging, call, payment) and includes a "New Session" form with scenario type selection and URL input.

3.6 QA Success Criteria

Criteria	Test	Expected Result
----------	------	-----------------

Dual sandbox coordination	Run a 2-agent scenario	Orchestrator provisions 2 browser instances and both reach "ready" state
Sync barrier	Barrier with 2 agents	Both agents block until the other arrives, then both resume
Event bus	Agent A signals Agent B	Agent B receives the event within the polling interval
Graceful teardown	Abort mid-scenario	All browsers close, session status set to "failed", no orphan instances
Sequential fallback	Memory-constrained environment	Falls back to running agents sequentially instead of parallel

Table 3: M25 QA Success Criteria

4. M26 - Cross-Device Messaging and Notification Testing

4.1 Objective

Build a cross-device messaging test agent that leverages the M25 orchestrator to verify real-time messaging functionality between two browser sessions. Agent A (sender) composes and sends messages while Agent B (receiver) monitors for incoming messages and verifies content, ordering, and delivery timing. This milestone also tests notification delivery, ensuring that push notifications or in-app alerts are correctly triggered when a message is received, including when the receiving application is in a background or minimized state.

4.2 Test Scenarios

The messaging test agent supports the following scenario types, each designed to validate a different aspect of cross-device communication reliability. Each scenario defines a sequence of actions for the sender and receiver agents, along with synchronization points that ensure proper coordination between the two browser sessions.

- **Basic Send/Receive:** Agent A sends a text message; Agent B verifies receipt and content match. Tests fundamental message delivery pipeline.
- **Multiple Messages:** Agent A sends 5 sequential messages; Agent B verifies all arrive in correct order. Tests ordering guarantees and message queuing.
- **Different Message Types:** Agent A sends text, image, file, and emoji messages; Agent B verifies each type renders correctly. Tests content type handling.

- **Typing Indicator:** Agent A starts typing; Agent B observes the typing indicator; Agent A stops; Agent B verifies it disappears. Tests real-time presence features.
- **Read Receipt:** Agent A sends a message; Agent B opens and reads it; Agent A sees the "read" receipt. Tests delivery confirmation flow.
- **Offline Queue:** Agent B goes offline (network disconnect simulation); Agent A sends messages; Agent B comes back online; Agent B receives queued messages. Tests offline resilience.
- **Notification Delivery:** Agent A sends a message; Agent B (background tab) receives a browser notification; Agent B clicks notification and navigates to message. Tests push notification integration.

4.3 Data Model Extension

The **OrchestratedSession** model is extended with a **messagingResults** JSON field that captures messaging-specific metrics. This includes delivery latency (time from send to receive), ordering correctness (percentage of messages arriving in order), content fidelity (percentage of messages with matching content), typing indicator accuracy, and read receipt latency. These metrics feed into the LLM analysis to generate a messaging reliability score and identify specific failure patterns.

```
messagingScore Float? // 0-100 messaging reliability score
deliveryLatency Json? // { avg: 120, p95: 340, p99: 580 } in ms
orderingCorrect Float? // Percentage of messages in correct order
contentFidelity Float? // Percentage of messages with matching content
notificationScore Float? // 0-100 notification delivery score
```

4.4 Core Agent: Messaging Tester

The **MessagingTester** class in **src/lib/agent/messaging-tester.ts** extends the orchestrator with messaging-specific logic. It defines action templates for common messaging operations (compose, send, verify receipt, check typing indicator, verify notification) and implements scenario-specific coordination patterns. The tester uses the 3-tier LLM fallback to analyze message delivery behavior: the LLM receives the raw action results from both agents and evaluates whether the messaging system is functioning correctly, identifying issues like delayed delivery, lost messages, or broken notifications.

Key methods include: **runMessagingScenario(url, scenarioType)** which orchestrates a complete messaging test; **composeMessage(agent, message)** which types a message in the compose field; **sendMessage(agent)** which clicks the send button; **verifyReceipt(agent, expectedContent, timeout)** which waits for a message matching the expected content; **checkTypingIndicator(agent, expected)** which verifies the typing indicator state; and **verifyNotification(agent, expectedTitle)** which checks for browser notification delivery. Each method returns a structured result with pass/fail status, timing data, and screenshot evidence.

4.5 API Routes

Method	Route	Description
POST	/api/orchestrator/messaging	Run a messaging test scenario (deducts credits)
GET	/api/orchestrator/messaging/results	List messaging test results with filtering
GET	/api/orchestrator/messaging/results/[id]	Get detailed messaging test result with agent logs

Table 4: M26 API Routes

4.6 UI Component: Messaging Test Panel

The **MessagingTestPanel** component visualizes cross-device messaging test results. It features a dual-column layout showing the sender and receiver timelines side by side, with connecting arrows indicating message delivery events. Key metrics (delivery latency, ordering correctness, content fidelity) are displayed as score circles at the top. The timeline view shows each action with timestamps, and message delivery events include a latency badge showing the time difference between send and receive. Failed or delayed deliveries are highlighted with warning indicators. The panel also includes a scenario type selector and a "Run New Test" button that opens a form with URL input and scenario selection.

4.7 QA Success Criteria

Criteria	Test	Expected Result
Message delivery	Agent A sends message	Agent B receives the message with matching content
Ordering	5 sequential messages	All messages arrive in correct order on receiver side
Typing indicator	Agent A types	Agent B sees typing indicator within 2 seconds
Notification	Background tab receives message	Browser notification appears with correct content
Offline queue	Receiver offline during send	Messages delivered when receiver comes back online

Table 5: M26 QA Success Criteria

5. M27 - Call Flow and Background State Testing

5.1 Objective

Implement a call flow testing agent that validates voice and video call workflows across two browser instances. The caller agent initiates a call, the callee agent receives and answers it, and both agents verify the call state transitions (ringing, connected, on hold, ended) and audio/video stream integrity. This milestone also tests background state behavior: simulating scenarios where the application is minimized or the device screen is "off" (simulated via page visibility API and reduced resource allocation), and verifying that notifications, call alerts, and state synchronization function correctly in these conditions.

5.2 Test Scenarios

- **Basic Call Flow:** Caller dials, callee rings, callee answers, both connected for 10 seconds, caller hangs up. Tests complete call lifecycle.
- **Call Rejection:** Caller dials, callee rings, callee declines. Tests rejection flow and state cleanup.
- **Call Timeout:** Caller dials, callee does not answer within timeout. Tests auto-disconnect and missed call notification.
- **Audio Verification:** During connected call, verify audio streams are active using WebRTC stats API and audio level detection. Tests media stream integrity.
- **Background Call Alert:** Callee app is in background state; caller dials; verify push notification/call alert appears and tapping it restores the app. Tests background notification delivery.
- **Screen-Off Notification:** Simulate screen-off state; incoming message triggers notification; verify notification content and tap-to-open behavior. Tests notification delivery in inactive states.
- **Call Hold/Resume:** Caller puts call on hold, callee sees hold indicator, caller resumes. Tests mid-call state transitions.

5.3 Data Model Extension

The **OrchestratedSession** model receives additional fields for call flow results. The call-specific metrics include call setup latency (time from dial to connected), ring duration, call duration accuracy, audio stream health (percentage of time audio is flowing), and background notification delivery rate. These metrics are computed by analyzing the WebRTC stats collected from both browser instances during the call, along with timing data from the action sequence logs.

```
callScore Float? // 0-100 call reliability score
callSetupLatency Json? // { avg: 850, p95: 2100 } in ms
audioStreamHealth Float? // Percentage of call with active audio
backgroundNotifScore Float? // 0-100 background notification score
callResults Json? // Per-scenario call results
```

5.4 Core Agent: Call Flow Tester

The **CallFlowTester** class in `src/lib/agent/call-flow-tester.ts` implements call-specific orchestration logic. It extends the base orchestrator with call action templates (dial, wait for ring, answer, verify audio, hang up, put on hold) and synchronization patterns specific to call flows. The tester leverages Puppeteer `page.evaluate()` to access WebRTC APIs in the browser context, collecting `RTCPeerConnection` stats to verify that audio and video tracks are active and healthy during the call.

For background state simulation, the tester uses Puppeteer to minimize the browser tab (triggering the Page Visibility API hidden state), simulate reduced CPU allocation via throttling, and then trigger incoming events (calls, messages) to verify that the application correctly handles these states. The tester checks for browser notification API usage, service worker registration, and push notification delivery. Audio verification uses a combination of WebRTC stats (bytes sent/received, audio level) and, where available, the `AudioContext` API to detect active audio streams.

5.5 API Routes

Method	Route	Description
POST	/api/orchestrator/call-flow	Run a call flow test scenario (deducts credits)
GET	/api/orchestrator/call-flow/results	List call flow test results
GET	/api/orchestrator/call-flow/results/[id]	Get call flow test detail with audio stats

Table 6: M27 API Routes

5.6 UI Component: Call Flow Test Panel

The **CallFlowTestPanel** component visualizes call flow test results with a call timeline view. The timeline shows state transitions (idle, dialing, ringing, connected, on-hold, ended) as colored segments for both caller and callee. Audio stream health is displayed as a waveform visualization showing audio levels over time, with gaps indicating dropped audio. Background state test results appear as separate cards showing notification delivery status and tap-to-open verification. The panel includes overall call reliability score, setup latency metrics, and an expandable log of WebRTC stats collected during the test.

5.7 QA Success Criteria

Criteria	Test	Expected Result
Call flow	Caller dials, callee answers	Both agents transition to "connected" state
Audio verification	Connected call for 10 seconds	WebRTC stats show active audio streams on both sides

Call rejection	Callee declines call	Both agents return to idle state, no zombie connections
Background alert	Incoming call while backgrounded	Notification appears, tapping it restores the app
Call hold	Caller holds, then resumes	Callee sees hold indicator, audio resumes after unhold

Table 7: M27 QA Success Criteria

6. M28 - Payment Flow Testing

6.1 Objective

Implement a payment flow testing agent that navigates complete e-commerce checkout flows and verifies payment processing using Stripe test mode. The agent tests the full payment lifecycle: cart addition, checkout form navigation, payment submission with various Stripe test cards (success, decline, error, 3D Secure), order confirmation verification, and webhook delivery validation. All testing is strictly confined to the Stripe test environment using designated test card numbers, ensuring no real financial transactions occur. The agent verifies that the application correctly handles all payment outcomes, including successful charges, declined cards, authentication-required scenarios, and network errors.

6.2 Test Scenarios

- **Successful Payment:** Navigate to product, add to cart, proceed to checkout, fill payment form with Stripe test card 4242424242424242, submit, verify order confirmation page. Tests the happy path.
- **Declined Card:** Same flow but use test card 4000000000000002 (decline). Verify error message is displayed, cart is preserved, and user can retry. Tests decline handling.
- **Insufficient Funds:** Use test card 40000000000009995. Verify specific error message about insufficient funds. Tests error specificity.
- **3D Secure Authentication:** Use test card 4000002500003155. Verify 3DS modal appears, complete authentication, verify payment completes. Tests SCA compliance.
- **Network Error:** Use test card 4000000000000119 (processing failure). Verify graceful error handling without exposing internal errors. Tests resilience.
- **Webhook Verification:** After successful payment, verify Stripe webhook is received and processed by polling the application webhook endpoint or checking database state. Tests webhook integration.

- **Multiple Currency:** Process payments in different currencies (USD, EUR, KES). Verify currency display and conversion. Tests multi-currency support.

6.3 Data Model Extension

The **OrchestratedSession** model receives payment-specific fields. These track the payment flow score, per-scenario results (including the Stripe test card used, the expected outcome, the actual outcome, and any error messages), checkout completion rate, and webhook delivery latency. The payment results are particularly sensitive from a security perspective, so all card numbers are stored using only Stripe-designated test card values (never real card numbers), and the results explicitly mark the test environment context.

```
paymentScore Float? // 0-100 payment reliability score
checkoutCompletionRate Float? // Percentage of checkouts reaching confirmation
webhookDeliveryMs Json? // { avg: 450, p95: 1200 } webhook latency
paymentResults Json? // Per-scenario payment outcomes
```

6.4 Core Agent: Payment Flow Tester

The **PaymentFlowTester** class in `src/lib/agent/payment-flow-tester.ts` implements payment-specific orchestration. Unlike messaging and call testing which require two agents, payment testing typically uses a single agent (the payer) navigating through the checkout flow. However, the orchestrator pattern is still used to support scenarios where a merchant dashboard needs to be verified simultaneously (e.g., checking that the order appears in the merchant admin panel after payment). The tester includes specialized action templates for e-commerce navigation: **addToCart(selector)**, **proceedToCheckout()**, **fillShippingAddress(address)**, **fillPaymentDetails(cardNumber, expiry, cvc)**, **submitPayment()**, **verifyConfirmation(expectedText)**, and **verifyWebhook(expectedEventType, timeout)**.

Stripe test card management is handled by a dedicated **StripeTestCards** utility that maps scenario types to the appropriate test card numbers and expected outcomes. This ensures that the agent always uses the correct test card for each scenario and that the verification logic knows what outcome to expect. The utility also handles 3D Secure authentication flows by detecting the Stripe 3DS iframe and completing the authentication step programmatically.

6.5 API Routes

Method	Route	Description
POST	/api/orchestrator/payment	Run a payment flow test scenario (deducts credits)
GET	/api/orchestrator/payment/results	List payment flow test results

GET	/api/orchestrator/payment/results/[id]	Get payment test detail with per-scenario outcomes
-----	--	--

Table 8: M28 API Routes

6.6 UI Component: Payment Flow Test Panel

The **PaymentFlowTestPanel** component displays payment test results with a checkout flow visualization. Each scenario is shown as a horizontal pipeline: Cart -> Shipping -> Payment -> Confirmation, with each step colored green (pass), red (fail), or yellow (warning). Failed steps expand to show the error message and the Stripe test card used. The panel includes an overall payment reliability score, checkout completion rate, and webhook delivery latency metrics. Scenario cards show the test card type (success, decline, 3DS, etc.), the expected vs actual outcome, and screenshots of key steps. A "Run Payment Test" button opens a form where users specify the target URL and select which scenarios to run.

6.7 QA Success Criteria

Criteria	Test	Expected Result
Successful payment	Test card 4242...4242	Checkout completes, confirmation page displayed
Declined card	Test card 4000...0002	Error message shown, user can retry, cart preserved
3D Secure	Test card 4000...3155	3DS modal appears, authentication completes, payment succeeds
Webhook verification	After successful payment	Webhook received and processed within 5 seconds
Error handling	Test card 4000...0119	Graceful error message, no stack traces exposed

Table 9: M28 QA Success Criteria

7. Architecture Deep-Dive

7.1 Multi-Sandbox Coordination Pattern

The multi-sandbox coordination pattern follows a coordinator-worker architecture adapted for serverless execution. A single serverless invocation acts as the coordinator, responsible for creating the orchestrated session record, provisioning sandbox instances, and managing the overall workflow. Each sandbox instance runs within the same invocation (when memory allows) or is delegated to a separate invocation that polls the database for its assigned actions. The coordinator uses a phase-based execution model: each scenario is broken into phases (setup, action, verification, teardown), and phases execute sequentially while actions within a phase can run in parallel across sandboxes.

The key architectural decision is the trade-off between parallelism and resource usage. On Vercel, a single serverless function can typically run 2 browser instances simultaneously within memory limits. For 3-4 agent scenarios, the orchestrator uses a hybrid approach: the first two agents run in parallel within the initial invocation, and subsequent agents are queued and run after the first pair completes. The database event bus ensures that agents running in different invocations can still synchronize through the shared SyncEvent table. This design provides a practical balance between the real-time coordination requirements of multi-device testing and the constraints of serverless infrastructure.

7.2 Synchronization Protocol

The synchronization protocol defines three primitive operations that sandboxes use to coordinate: **barrier**, **signal**, and **state update**. A barrier is a named synchronization point where all agents must arrive before any can proceed. When an agent reaches a barrier, it writes a SyncEvent with type "barrier" and the barrier name. The coordinator (or the agent itself) polls for barrier events from all agents with that name. Once all expected agents have arrived, a release event is written, and all waiting agents resume execution.

A signal is a directed event from one agent to another (or broadcast to all). It carries a payload and is consumed by the target agent. Signals are used for actions like "message sent" (Agent A signals Agent B that a message has been sent) or "call answered" (Callee signals Caller that the call was answered). The consuming agent polls for unconsumed events matching its target role and the expected event type, with exponential backoff and a configurable timeout. State updates allow agents to share mutable state through the database, enabling scenarios where one agent needs to know the current state of another (e.g., "Agent B is now on the chat screen").

7.3 Event Bus Design

The SyncEvent table implements the event bus with the following design properties: events are write-once (an event is created but never modified), consumption is idempotent (checking for an event multiple times is safe), and ordering is guaranteed by sequential IDs within a session. The consumed flag and consumedAt timestamp support exactly-once consumption semantics: an agent marks an event as consumed atomically when it processes it, preventing duplicate handling. The table is indexed on (sessionId, eventType) for efficient event type filtering and (sessionId, targetAgent, consumed) for efficient agent-specific event polling.

Event polling uses an exponential backoff strategy starting at 200ms, increasing to a maximum of 2 seconds, with a total timeout configurable per scenario (default 30 seconds). This balances responsiveness (events are detected quickly when they occur) with database load (polling frequency decreases over time for long waits). For scenarios where the coordinator runs both agents within the same invocation, polling can be replaced with direct in-process signaling for lower latency, falling back to database polling only for cross-invocation scenarios.

7.4 Credit Cost Model

Multi-device testing incurs higher resource costs than single-browser testing due to multiple browser instances, longer execution times, and increased LLM usage for cross-agent analysis. The credit cost model reflects these factors with a new credit action **orchestrated_test** that costs 12 credits per session (compared to 2 credits/min for single test execution). This covers the cost of provisioning 2+ browser instances, running the orchestration logic, performing LLM analysis on cross-agent results, and storing the extended result set including sync events and per-agent logs.

Action	Credits	Est. Cost	Coverage
Orchestrated Test	12	\$0.40	Multi-sandbox session (2+ agents)
Messaging Test	12	\$0.40	Includes orchestrated test + messaging analysis
Call Flow Test	15	\$0.50	Includes orchestrated test + audio verification
Payment Flow Test	15	\$0.50	Includes orchestrated test + webhook verification

Table 10: Phase 5 Credit Cost Model

8. Prisma Schema Additions Summary

Phase 5 introduces three new Prisma models and extends the existing Project model with new relations. The complete schema additions are summarized below. All new models follow the established patterns: cuid-based IDs, timestamp tracking with createdAt/updatedAt, JSON fields for flexible result storage, and comprehensive indexing for query performance. The SyncEvent model includes a cascade delete relation to OrchestratedSession, ensuring that event bus data is cleaned up when sessions are removed.

Model	Key Fields	Relations	Milestone
OrchestratedSession	scenarioType, status, overallScore, messagingScore, callScore, paymentScore	Project, SandboxInstance[], SyncEvent[]	M25

SandboxInstance	agentRole, browserId, status, actions, results, score	OrchestratedSession	M25
SyncEvent	eventType, sourceAgent, targetAgent, payload, consumed	OrchestratedSession	M25

Table 11: Prisma Schema Additions

9. API Route Map

Phase 5 adds 13 new API routes organized under the `/api/orchestrator/` prefix. All routes follow the established authentication pattern (NextAuth session required) and credit deduction model (check balance, deduct, execute, settle on failure). The routes are grouped by milestone, with M25 providing the foundational session management endpoints and subsequent milestones adding scenario-specific endpoints.

Route	Method	Milestone	Credits
<code>/api/orchestrator/sessions</code>	POST	M25	12
<code>/api/orchestrator/sessions</code>	GET	M25	0
<code>/api/orchestrator/sessions/[id]</code>	GET	M25	0
<code>/api/orchestrator/sessions/[id]/abort</code>	POST	M25	0
<code>/api/orchestrator/messaging</code>	POST	M26	12
<code>/api/orchestrator/messaging/results</code>	GET	M26	0
<code>/api/orchestrator/messaging/results/[id]</code>	GET	M26	0
<code>/api/orchestrator/call-flow</code>	POST	M27	15
<code>/api/orchestrator/call-flow/results</code>	GET	M27	0
<code>/api/orchestrator/call-flow/results/[id]</code>	GET	M27	0
<code>/api/orchestrator/payment</code>	POST	M28	15
<code>/api/orchestrator/payment/results</code>	GET	M28	0
<code>/api/orchestrator/payment/results/[id]</code>	GET	M28	0

Table 12: Phase 5 API Route Map

10. UI Component Inventory

Phase 5 introduces four new dashboard components, each following the established pattern of score circles, expandable result cards, severity badges, and action forms. The components are integrated into the project detail page alongside existing panels for security, accessibility, and media verification. The orchestrator panel serves as the primary entry point, while the messaging, call flow, and payment panels provide scenario-specific visualizations that are accessible both from the orchestrator panel and independently from the project detail page.

Component	File	Milestone	Key Features
OrchestratorPanel	orchestrator-panel.tsx	M25	Session timeline, sandbox cards, sync event log, new session form
MessagingTestPanel	messaging-test-panel.tsx	M26	Dual-column sender/receiver timeline, delivery metrics, scenario selector
CallFlowTestPanel	call-flow-test-panel.tsx	M27	Call state timeline, audio waveform, background notification cards
PaymentFlowTestPanel	payment-flow-test-panel.tsx	M28	Checkout pipeline visualization, test card results, webhook latency

Table 13: Phase 5 UI Components

11. Dependency and Risk Analysis

11.1 Dependencies

Phase 5 has both internal and external dependencies that must be managed carefully. Internally, all four milestones depend on the M25 orchestrator infrastructure being completed first. M26, M27, and M28 can be developed in sequence but share the same underlying orchestration layer. The existing Puppeteer browser automation infrastructure (4-tier provider chain from M3) is critical, as Phase 5 requires reliable multi-instance browser provisioning. The Prisma/Neon PostgreSQL stack must support the increased write load from sync event polling, which could generate hundreds of database operations per orchestrated session.

External dependencies include: Browserless cloud for reliable browser instance provisioning (the existing tier-1 provider), Stripe test mode for M28 payment testing (free tier available), and the z-ai-web-dev-sdk for LLM-powered analysis of cross-agent results. Each of these has been previously integrated in earlier phases, but Phase 5 stresses them more heavily: Browserless must handle 2-4 concurrent instances per test, the LLM must analyze larger result sets (2+ agent logs combined), and

Stripe must be configured for test mode webhook delivery to the Probato development environment.

11.2 Risk Assessment

Risk	Impact	Likelihood	Mitigation
Browser instance memory exhaustion	High	Medium	Limit to 2 concurrent instances; sequential fallback; memory monitoring
Sync event polling latency	Medium	High	Exponential backoff; in-process signaling for same-invocation agents
Serverless timeout on long scenarios	High	Medium	Phase-based execution; DB state continuation across invocations
Stripe test mode limitations	Low	Low	Test card coverage is comprehensive; document any edge cases
WebRTC API availability in Puppeteer	Medium	Low	Use Sparticuz Chromium with WebRTC support; fallback to behavioral testing
Database load from sync polling	Medium	Medium	Index optimization; batch event reads; connection pooling via Prisma

Table 14: Risk Assessment

12. Timeline

Phase 5 spans eight weeks with four two-week milestones. Each milestone follows the established development pattern: model definition and schema migration in the first day, core agent implementation over the next five days, API routes and UI components for three days, then testing, bug fixes, and deployment. The milestones are sequential due to the dependency on the M25 orchestrator, though some preparatory work on M26-M28 action templates and UI designs can begin in parallel once the M25 data models are finalized.

Week	Milestone	Focus	Deliverables
1-2	M25	Multi-Sandbox Orchestrator	OrchestratedSession + SandboxInstance + SyncEvent models; MultiDeviceOrchestrator agent; 4 API routes; OrchestratorPanel

3-4	M26	Cross-Device Messaging	MessagingTester agent; 7 messaging scenarios; 3 API routes; MessagingTestPanel; messaging metrics
5-6	M27	Call Flow + Background	CallFlowTester agent; 7 call scenarios; 3 API routes; CallFlowTestPanel; audio verification; background state sim
7-8	M28	Payment Flow	PaymentFlowTester agent; 7 payment scenarios; 3 API routes; PaymentFlowTestPanel; Stripe test cards; webhook verification

Table 15: Phase 5 Timeline

Upon completion of Phase 5, Probato will be the only AI-powered testing platform capable of orchestrating multi-device test scenarios in a serverless environment. This capability differentiates Probato from single-browser testing tools and positions it for real-world testing of collaborative applications, communication platforms, and e-commerce systems where cross-device behavior validation is essential.